Migrating from Elasticsearch 8 to 9 and away from Enterprise Search

Why we need to migrate

Elastic is **removing Enterprise Search entirely** in Elasticsearch 9. KISS currently depends on it for three things:

- **A search recipe** — Enterprise Search tells KISS how to search: which fields to look in and how much weight to give each one
- **Grouping data sources** — it organises all content (VAC, Kennisbank, websites, …) into one searchable unit
- **Website crawling** — it crawls websites and stores the content for searching

All three need a replacement before we can upgrade to ES9.

> At runtime, KISS only calls Enterprise Search for one thing: fetching the search recipe.
> Everything else already goes directly to Elasticsearch.

How search works in KISS today

When a user searches, three steps happen:

1. **KISS asks Enterprise Search:** "*How should I search for this?*"
   It returns a set of search instructions — which fields to search, how much weight each gets, and how broad or strict the matching should be.

2. **KISS adjusts those instructions:**
   Adds filters (e.g. show only VAC), ranking rules (website results rank lower), and autocomplete hints.

3. **KISS sends the final search to Elasticsearch**, which runs it and returns results.

All structured content (VAC, Kennisbank, Smoelenboek, SharePoint) is already stored directly in Elasticsearch — Enterprise Search only *registers* them. The actual data storage and retrieval has never depended on it.

Changes

What	Current state	After migration
VAC, Kennisbank, Smoelenboek, SharePoint	Already stored directly in Elasticsearch	No change to how data is stored
Sync tool (KISS-Elastic-Sync)	Calls Enterprise Search only to *register* indices	Remove one file, two function calls
Search recipe	Fetched from Enterprise Search at runtime	Stored as configuration; applied by the server
Website crawler	Built into Enterprise Search	Replace with Elastic's standalone Open Crawler
Relevance / precision tuning	Admin screens in Kibana	Export current settings first; rebuild UI later (optional)

Suggestion: merge the sync tool into this repository

KISS-Elastic-Sync lives in a separate repository, but it is part of the same product — same team, same release cycle.

The problem during this migration:

- Changes span both codebases significantly — two repos means two PRs, two reviews, two releases
- AI tools work best with the full picture in one place
- A developer fixing a search issue needs to trace code across two repositories

The proposal:
Move the sync tool code into the KISS repository.
Deployment stays separate — it still runs as a scheduled job, not alongside the web server.
Git history can be preserved with `git subtree` or `git filter-repo`.

> **Recommendation:** merge first, then do the migration.
> The scope of sync tool changes during this migration makes the two-repo friction immediately painful.

Design choice: where does the search logic live?

Right now the **frontend** builds the full search request. There are two options.

Option A — Keep it in the browser *(less work)*

- ✅ Least migration effort
- ❌ Full search configuration visible to anyone who opens the browser dev tools
- ❌ Authenticated users can send arbitrary search requests

Option B — Move it to the server *(recommended)*

The browser sends simple parameters. The server builds the search request, applies weights, and talks to Elasticsearch.

- ✅ Search configuration stays on the server, not visible in the browser
- ✅ Cleaner frontend code
- ❌ More migration work — new server-side search component needed

Relevance Tuning UI

This is **independent of Option A vs B**. It can be combined with either.

After migration, the Kibana tuning screens disappear. The question is what replaces them.

Option: build a simple admin page in KISS where authorised users can adjust:

- Which fields matter most and by how much *(e.g. "title is 3× more important than body text")*
- How strict the search is *(broad / normal / strict)*

> **Recommendation:** not a blocker for the core migration.
> First, save the current weights and precision level as configuration.
> Build the UI as a follow-on once the core migration is stable.

Precision & relevance tuning — and our history

Relevance tuning — which fields matter most (weights per field).
Precision tuning — how strict the search is (slider 1–11: broad → strict).

Both are set via Kibana and baked into the search recipe from Enterprise Search.

Precision	Behaviour
1–2 (default)	Broad — includes loose matches and typos
3–8	Increasingly strict
9–11	Only close matches; all terms must appear

- Customers were **unhappy with the default (level 2)** — results felt noisy
- We experimented with **level 8** — noticeably better for our content
- After migration, precision should be an **explicit config value**, not hidden in a recipe snapshot

An opportunity to improve search quality

The current setup was built around App Search's generic defaults — designed for any content in any language.

What this means:

- The setup supports 11 precision levels with different matching rules for each — KISS uses one level at a time, so most of that complexity is unused
- Field weights have likely never been reviewed against real search results
- A purpose-built Dutch search configuration may work better than inherited generic defaults

Our approach to improving quality

> Migrate first, improve after.

1. **Reproduce current behavior** in native Elasticsearch — safe, low-risk, preserves what works today
2. **Improve iteratively** — with real user feedback as the measure, not inherited defaults

Moving search logic to the server (Option B) makes iteration much easier: tuning changes don't require a frontend release.

Before touching anything — capture the current state

The most important first step:

Ask Enterprise Search: "*What search recipe are you currently using?*"
Save the complete answer. This gives us:

- All **field weights** (what Relevance Tuning is currently set to)
- The **precision level** in use (what Precision Tuning is currently set to)
- The **list of all data sources** being searched

Migration phases

Phase	Work
1 — Prepare	Capture search recipe · set up new crawler on staging
2 — Server search *(Option B)*	Build server-side search component · apply field weights · verify quality
3 — Update browser	Remove Enterprise Search calls · connect to new server endpoints
4 — Update sync tool	Remove Enterprise Search calls · configure Open Crawler
5 — Clean up	Remove Enterprise Search service · delete old indices · update config

Replacing the web crawler

	Enterprise Search Crawler	Elastic Open Crawler
Where it runs	Built into Enterprise Search	Separate service (scheduled job)
Where content lands	Hidden internal Elasticsearch index	Regular Elasticsearch index
Status	Discontinued in ES9	Official Elastic replacement (currently in beta)

Recommended approach

Decision	Recommendation
Repositories	Merge KISS-Elastic-Sync into this repo first
Architecture	Option B — move search logic to the server
Crawler	Replace with Elastic Open Crawler
Precision & weights	Capture from live system; store as explicit config
Search quality	Migrate with same behavior first, then improve iteratively
Relevance Tuning UI	Follow-on feature after core migration is stable

Component overview

Component	Change
Enterprise Search	Remove entirely
Web crawler	Replace with Open Crawler
Sync tool	Remove Enterprise Search calls
Backend server	Add search logic (Option B)
Frontend code	Simplify, remove recipe fetching
Search config	Export current recipe; store as config